CPSC 315 · FINAL PROJECT

Smart Library Management System

Nine modules, from project plan to working system

Michael Pelitera

What the system does

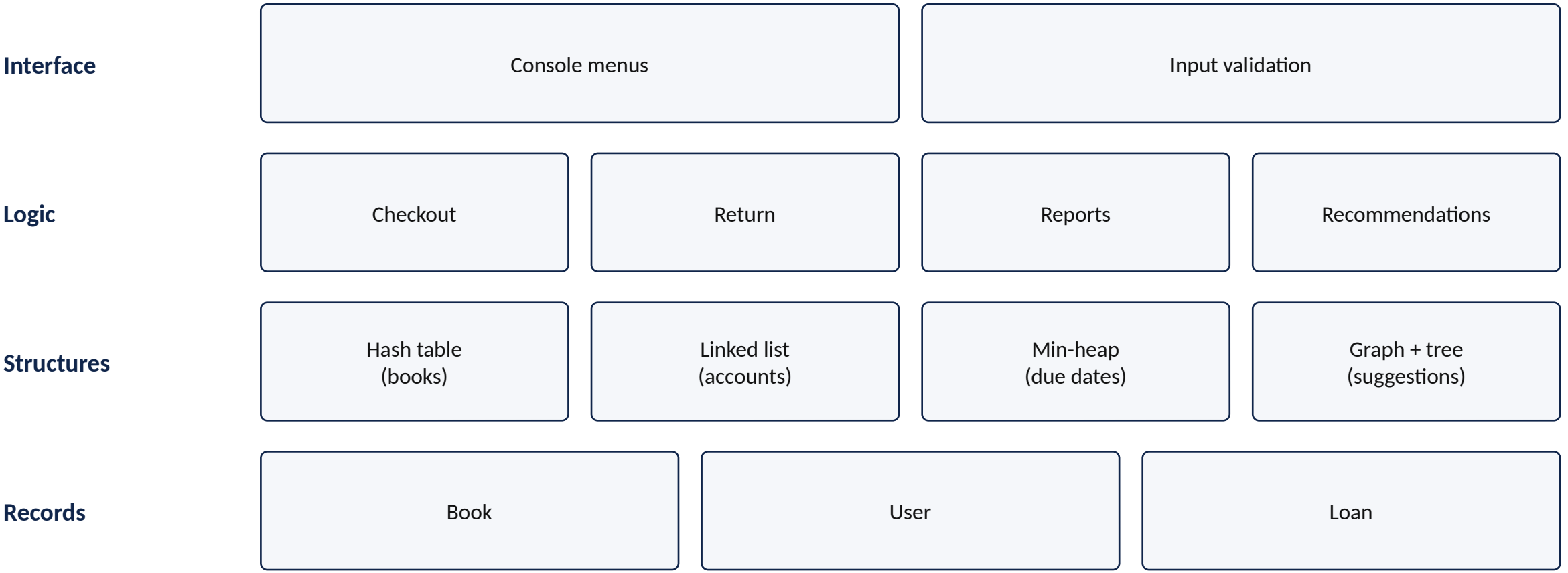
Three people, three different jobs, one catalog.

Patron	Finds a book, borrows it, sees what is due and when. Searches by part of a title or author. Does not need an ISBN.
Librarian	Checks items back in and works the overdue list. Sees which items are late, by how long, and who has them.
Administrator	Looks at the collection as a whole. Counts titles, copies, and what is currently out.

Nobody using the system needs to know what a hash table is. That is the test the interface has to pass.

How it is put together

Four layers. Each one only talks to the layer below it.



The payoff: a structure can be swapped without touching anything above it, and the logic can be tested without typing on a keyboard.

Live demo

Six scenes, about three minutes. Scripted so it runs the same way every time.

1. Search and borrow
2. A refusal that explains itself
3. Recommendations
4. Account view
5. Overdue report (librarian)
6. Inventory report

```
 SEARCH CATALOG            Mike / Student
----------------------------------------
 Title or author: apollo

  1. Digital Apollo        3 available
  2. The Apollo Guidance   all copies out

  2 titles found.
  3. Back to menu

 Select [1-3]: 1
   Checked out: Digital Apollo
   Due in 21 days (day 121).
```

```
 OVERDUE REPORT       A. Rivera / Librarian
-----------------------------------------
 As of day 100

   Design Patterns    Priya     9 days late
   The Lean Startup   Sam       4 days late

   2 overdue items.
```

Deterministic: three consecutive runs produce byte-identical output. A recorded run stands by as the fallback.

Why these structures

Each one was chosen for the question it answers quickly.

The question	What answers it	Why it is the right fit
Where is this exact book?	Hash table on ISBN	One jump to the record, no matter how big the catalog gets.
What is due soonest?	Min-heap on due date	Earliest date always sits on top, so overdue items never require a full scan.
Who is this patron?	Linked list + index	The list keeps accounts in order; the index finds one without walking it.
What else might they like?	Graph of co-borrowing	Captures which titles get read together, which a list or tree cannot express.
Which suggestion is best?	Tree of scores	Puts candidates in ranked order, which a graph has no way to do.

The honest version of the lesson: the structure with the better textbook complexity is not automatically the better choice. Binary search beats a hash table on paper and lost here, because keeping the catalog sorted costs more than it saves.

What testing actually found

Two defects that passed all sixty of the original checks.

Phantom overdue notice

One patron, two copies of the same title.

The loan lookup stored a single loan ID per patron and title, assuming nobody could hold two copies. Nothing enforced that. The second checkout overwrote the first, leaving a deadline entry that could never be cleared. Both copies came back and the system still reported one overdue.

One-way connections

Trimming the recommendation graph.

Each title keeps its 50 strongest links. The trim removed dropped links from one end only, so the other title kept pointing back at something that had forgotten it.

Every component was individually correct in both cases. Suite is now at 72 checks, with a regression test holding each failure in place.

What it does not do

Prepared in advance so these are answers, not surprises.

Nothing is saved

Everything lives in memory and is gone when the program closes.

Write the records out on exit and read them back at startup.

One person at a time

Two simultaneous checkouts of a last copy could both succeed.

One lock covering the account list and its index together.

No holds, fines, or renewals

Designed in Module 2, never built.

A queue per title plus a balance on the account.

No compression

Huffman coding is in the project spec and is not implemented.

Only meaningful once there is a file on disk to compress.

Each of these is written up in the technical report with what closing it would take.

How it got built

Nine modules, each one depending on the last.

1-2	**Plan and design**	Requirements, architecture, UML, interface mock-ups.
3-4	**Core features**	Catalog and accounts, with the tests that go with them.
5	**Advanced features**	Borrowing, deadlines, and the recommendation engine.
6	**Testing**	Two defects found and fixed. Suite grew 60 to 72 checks.
7	**Documentation**	User guide and technical report. Writing them exposed four interface problems.
8	**Presentation**	This deck and a scripted demo.

The thing I did not expect: writing the user guide found more interface problems than testing did.